

DBMS WEEK 2 REPORT

Product and Category Management System

Introduction

The e-commerce company maintains information about products and their categories. Each product belongs to a specific category. The system manages product details such as product name, price, and stock availability.

Design of Category and Product Tables

Category Table

The Category table stores category information such as Category ID, Category Name and Description.

```
SQL> CREATE TABLE Category(Category_ID VARCHAR2(7),  
2 Category_Name VARCHAR2(20),  
3 Description VARCHAR2(50));  
  
Table created.
```

Product Table

The Product table stores product information such as Product ID, Product Name, Category ID, Price and Stock Quantity.

```
SQL> CREATE TABLE Product(Product_ID VARCHAR2(7),  
2 Product_Name VARCHAR2(20),  
3 Category_ID VARCHAR2(7),  
4 Price NUMBER(9,2),  
5 Stock_Quantity Number);  
  
Table created.
```

Relationships and Constraints

Relationship

One Category can have many Products.

Primary Key

Primary Key uniquely identifies each row in a table.

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);
```

Table altered.

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);
```

Table altered.

Foreign Key

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);
```

Table altered.

NOT NULL Constraint

NOT NULL ensures that a column cannot contain an empty value.

```
SQL> ALTER TABLE Category
  2  MODIFY Category_Name VARCHAR(100) NOT NULL;
```

Table altered.

```
SQL> ALTER TABLE Product
  2  MODIFY Product_Name VARCHAR(150) NOT NULL;
```

Table altered.

CHECK Constraint

CHECK validates whether the value satisfies a given condition.

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Price CHECK (Price > 0);

Table altered.

SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);

Table altered.
```

UNIQUE Constraint

UNIQUE ensures that duplicate values are not allowed.

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);

Table altered.
```

Insert Values

Category Table

```
SQL> INSERT INTO Category VALUES ('Cat0001','Mobiles','Mobile phones and accessories');

1 row created.
```

```
SQL> INSERT INTO Category VALUES ('Cat0002','Stationery','School and office stationery');

1 row created.
```

```
SQL> INSERT INTO Category VALUES ('Cat0003','Furniture','Home and office furniture');

1 row created.
```

```
SQL> INSERT INTO Category VALUES ('Cat0004','Footwear','Shoes and sandals');

1 row created.
```

```
SQL> INSERT INTO Category VALUES ('Cat0005','Kitchen','Kitchen items and utensils');

1 row created.
```

product Table

```
SQL> INSERT INTO Product VALUES ('P2001','Smartphone','Cat0001',28000,15);
1 row created.

SQL> INSERT INTO Product VALUES ('P2002','Smart Watch','Cat0001',4500,20);
1 row created.

SQL> INSERT INTO Product VALUES ('P2003','Power Bank','Cat0001',1200,25);
1 row created.

SQL> INSERT INTO Product VALUES ('P2004','Notebook','Cat0002',120,40);
1 row created.
```

Display Category Table

```
SQL> select * from category;
```

```
CATEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
Cat0001
```

```
Mobiles
```

```
Mobile phones and accessories
```

```
Cat0002
```

```
Stationery
```

```
School and office stationery
```

```
CATEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
Cat0003
```

```
Furniture
```

```
Home and office furniture
```

```
Cat0004
```

```
Footwear
```

```
CATEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
Shoes and sandals
```

```
Cat0005
```

```
Kitchen
```

```
Kitchen items and utensils
```

Display Product Table

```
SQL> select * from product;
```

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

P2001

Smartphone

Cat0001 28000 15

P2002

Smart Watch

Cat0001 4500 20

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

P2003

Power Bank

Cat0001 1200 25

P2004

Notebook

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

Cat0002 120 40

CRUD Operations

Insert Record

```
SQL> INSERT INTO Product (Product_ID,Product_Name,Category_ID,Price,Stock_Quantity) VALUES ('P3002','Washing Machine','Cat0003',18000,12);
```

1 row created.

Update Price

```
SQL> UPDATE Product SET Price = 60000 WHERE Product_Name = 'Smart Watch';
```

1 row updated.

Delete a Product

```
SQL> DELETE FROM Product WHERE Product_Name = 'Notebook';

1 row deleted.
```

Category-wise Product Reports

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price, p.Stock_Quantity FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID;
```

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK_QUANTITY
Mobiles	Smartphone	28000	15
	Smart Watch	60000	20
Mobiles	Power Bank	1200	25
	Washing Machine	18000	12

Count number of products in each category

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	PRODUCT_COUNT
Mobiles	3
Furniture	1

Find highest-priced product in each category

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID WHERE p.Price = (SELECT MAX(p2.Price) FROM Product p2 WHERE p2.Category_ID = p.Category_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Mobiles	Smart Watch	60000
Furniture	Washing Machine	18000

Display categories having more than 5 products

```
SQL> SELECT c.Category_Name FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID) > 5;
no rows selected
```

Find average product price category-wise

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
CATEGORY_NAME
-----
AVG_PRICE
-----
Mobiles
29733.3333
Furniture
18000
```

Conclusion

The Product and Category Management System was designed using SQL. Category and Product tables were created with appropriate primary key, foreign key, NOT NULL, CHECK and UNIQUE constraints. Product and category data were inserted and CRUD operations were performed. Finally, category-wise product reports were generated using SQL queries.